Assignment T5: Second Iteration

Published | Assign To | Edit | ⋮

This is a team assignment. See submission instructions at the end.

Implement the **final** version of your service. If it does not include (at least) the full API as described in your revised proposal, your README should explain what is different and why you decided to change it. If you added anything significant to the API that was not described in your revised proposal, explain what and why.

Develop a **demoable** client app that uses your service API and targets one or more of the designated target audiences. Make sure your service can interface to *multiple instances* of the client running concurrently and tell them apart. The code for your client app can be in the same repository as your service (**Google stores billions of lines of code in a single repository (https://cacm.acm.org/magazines/2016/7/204032-why-google-stores-billions-of-lines-of-code-in-a-single-repository/fulltext)**), but it's ok to put the client code in a separate repository as long as the README for your main repository provides a link to this second repository and all members of the teaching staff have access.

Update your README to describe what your client app does, how it uses your service, and how it helps its target audience. Explain what its users can do now with your app that they could not do before or better/faster/cheaper than they could before.

Implement end-to-end tests where your client exercises as much functionality of your service as possible. Although ideally automated, it's ok to run end-to-end tests manually. Document any manual tests with a checklist or some other mechanism to make sure you can re-run the exact same set of tests as needed, and include these instructions in your repo.

Also add to your README.md instructions explaining how to build, run and test your sample client with your service. All configuration files needed for build, run and test must be included in your repo. Describe what a third-party would need to do to develop their own client that uses your service.

The remainder of your toolchain, as discussed below, only needs to address the service and is optional for the client.

Add continuous integration (CI) to your github repository using github actions, so that all your build, analysis and testing tools run automatically for every commit. All CI configuration files should be included in your repository. Include some CI reports in your repository.

Implement *internal* integration tests between all assemblies of multiple units within your service that work together in some way, either by invoking each other or by sharing data. Add *external* integration tests checking how your code uses any external resources like files and databases and any third-party code

like libraries and services. Document each test to explain what it is integrating with what. All your integration tests should run automatically during CI.

Use a branch coverage tool together with your unit, integration and system testing during CI. Try to achieve at least 85% branch coverage. Try to fix most of the bugs found by the tests. Include some testing and coverage reports in your repository.

Use a static analysis bug finder tool on your entire codebase that runs automatically during CI. Try to fix most of the bugs found by the analyzer. Include some static analysis reports in your repository.

Update your unit test suite to support all the major "units" (methods, functions, procedures, subroutines, etc.) in your code. You should use a mocking framework if applicable. Unit tests should be grouped so related code (e.g., classes, modules, files, etc.) share setup, teardown and mocks when appropriate. Your entire collection of unit tests should run automatically during CI.

Update your API (system) test suite to exercise your service's complete API, e.g., using some API testing tool. Your tests should exercise the multiple clients and persistent data aspects of your service. Your API tests should also run automatically during CI.

Make sure all your test code and scripts have been committed to the main branch of your repository (its ok to have some other test code in branches but it will not be considered part of your submission). All main branch commit messages, except for the initial repo-setup commit(s), should say something meaningful about what was added/removed/changed. Your entire codebase, including test cases, should be compliant with a style checker appropriate for your language/platform. Include some recent reports from your style checker in your repository. Make sure to **tag** (https://stackoverflow.com/questions/18216991/create-a-tag-in-a-github-repository) the set of file revisions considered part of the second iteration.

Your repository's README.md should document all the operational entry points to your service and/or link to some external API documentation you wrote. Cover all the inputs and outputs, including status codes. If certain entry points must be called in certain orders or never called in certain orders, make sure to say so. Your README should also include instructions explaining how to build, run and test your service. All configuration files needed for build, run and test must be included in your repo.

If any third-party code is included in your codebase, your README should also document exactly which code this is, where it resides in your repository, and where you got it from (e.g., download url, ChatGPT prompts). However, in most cases you should use a package manager or build system to fetch third-party libraries as needed rather than bloating your codebase.

Submission instructions: One member of your team should submit a url pointing to your codebase repository. We encourage you to make your repository public, but if you prefer to keep it private then make sure the instructor and all the Mentors have collaborator access!

Points 50

Submitting a website url

Due	For	Available from	Until
Nov 30, 2023	Everyone	Oct 15, 2023 at 12am	Dec 6, 2023 at 11:59pm

Second Iteration Rubric	You've already rated students with this rubric. Any major changes could affect their assessment results.

Criteria	Ratings	Pts
Client Did they develop a demoable client that exercises a reasonable subset of their API and does the README document how to build the client and run the client together with their service?	This area will be used by the assessor to leave comments related to this criterion.	5 pts
Multiple Client Instances Does their service support multiple simultaneous instances of the sample client? Can the service tell them apart? Can the service handle multiple requests/responses from different client instances in parallel and track which requests involving which data came from which client (and send corresponding responses only to the appropriate clients)? Does the test suite for the service include tests that check that the service indeed operates properly when it has multiple concurrent clients?	This area will be used by the assessor to leave comments related to this criterion.	10 pts
Continuous Integration Does the main repo perform CI such that all build, analysis and testing tools are automatically run for every commit to the main branch of the codebase (or on a schedule such as nightly)? Are the CI reports in the repo or linked from the repo?	This area will be used by the assessor to leave comments related to this criterion.	5 pts
Internal Integration Tests Does the codebase include integration tests for the components of the service code that run automatically during CI?	This area will be used by the assessor to leave comments related to this criterion.	5 pts
Branch Coverage Did the team run branch coverage during automated testing? Does the repo contain or link to coverage reports? Is the branch coverage at least 85%? If 85% was not achieved, is there some explanation why not in the README or elsewhere in the repo?	This area will be used by the assessor to leave comments related to this criterion.	5 pts
Bug Finder Did the team run a static analysis 'bug finder' tool on their entire codebase (except for any third-party code)? Does it run automatically during CI? Are bug finder reports included in or linked from the repo?	This area will be used by the assessor to leave comments related to this criterion.	5 pts

Criteria	Ratings	Pts
End-to-End Testing Is there reasonable evidence that the team did substantial end-to-end testing of their client utilizing their service? It's ok if these tests are manual, but there should be a written checklist or some other mechanism that makes it easier to redo the exact same tests multiple times.	This area will be used by the assessor to leave comments related to this criterion.	6 pts
Third Party Code Do they document all third-party code used, whether library, API, database, etc., where it is in the repo (if applicable), and where they got it from (e.g., link to download or documentation)?	This area will be used by the assessor to leave comments related to this criterion.	1 pts
API Documentation Has their API documentation been updated to the 'final' version of their service?	This area will be used by the assessor to leave comments related to this criterion.	1 pts
Configuration Is sufficient documentation included in the repo to enable an arbitrary developer (who already knows their programming language and its ecosystem) to build, run and test their service? Are all necessary configuration files included in the repo?	This area will be used by the assessor to leave comments related to this criterion.	1 pts
Client Target Audience Does the client provide what seems like reasonable functionality for one or more of the designated target audiences?	This area will be used by the assessor to leave comments related to this criterion.	3 pts
External Integration Tests Are there tests for checking the interfaces to any third-party libraries and services used? Are there tests checking the interfaces to all environmental or external resources such as files and databases?	This area will be used by the assessor to leave comments related to this criterion.	3 pts
		Total Points: 50